

Laboratory 1: Neural Models — Learning, Depth, Activations, and Output Layers

AI Laboratory Report

Objective: study learning in a small neural model using XOR, investigate backpropagation and initialization symmetry, compare hidden activations, and extend the model to three-class classification.

Task 1: Problem Specification

Input space: two binary sensor inputs x_1 and x_2 .

Output space: one binary warning output. The warning is active exactly when one sensor is active and the other is inactive.

x_1	x_2	y
0	0	0
0	1	1
1	0	1
1	1	0

A single affine transformation followed by a sigmoid still produces a linear decision boundary, so it cannot represent XOR perfectly. A nonlinear hidden layer is therefore required.

Task 2: Model Design

Architecture: $2 \rightarrow 2 \rightarrow 1$

Hidden activation: sigmoid for the baseline experiment.

Binary output: one logit during training, converted to a probability with sigmoid for reporting.

Loss: BCEWithLogitsLoss.

Optimization: full-batch gradient-based optimization using SGD.

The hidden nonlinearity is necessary because stacking affine layers without nonlinear activations still collapses to a single affine map. Success was checked using loss reduction, correct predictions on all four XOR examples, and nonzero gradients.

Task 3: LLM-Assisted Implementation

Exact prompt used:

Generate minimal PyTorch code for the following model and dataset.

Dataset:
XOR training examples:
(0,0) → 0
(0,1) → 1
(1,0) → 1
(1,1) → 0

Requirements:
- Use a $2 \rightarrow 2 \rightarrow 1$ neural network.
- Use a sigmoid activation in the hidden layer.
- Use a sigmoid binary output.
- Use binary cross-entropy loss, or BCEWithLogitsLoss if using logits.
- Use random weight initialization.
- Use full-batch training.
- Train for a few thousand lightweight CPU steps.

- Set a random seed for reproducibility.
- Print the initial and final loss.
- After training, report the final probability for all four inputs.
- Print the thresholded predictions.
- After backward(), expose at least one parameter gradient tensor.
- Keep the implementation minimal and do not change the architecture or dataset.

Changes/corrections documented: (1) BCEWithLogitsLoss was used with raw logits for the binary training loss. (2) sigmoid was applied only after training when reporting probabilities and thresholded predictions.

Final Binary XOR Code

```
import torch
import torch.nn as nn

torch.manual_seed(42)

X = torch.tensor([
    [0.0, 0.0], [0.0, 1.0],
    [1.0, 0.0], [1.0, 1.0]
])
y = torch.tensor([
    [0.0], [1.0], [1.0], [0.0]
])

model = nn.Sequential(
    nn.Linear(2, 2),
    nn.Sigmoid(),
    nn.Linear(2, 1)
)

criterion = nn.BCEWithLogitsLoss()
optimizer = torch.optim.SGD(model.parameters(), lr=1.0)

with torch.no_grad():
    initial_loss = criterion(model(X), y).item()

for step in range(5000):
    logits = model(X)
    loss = criterion(logits, y)
    optimizer.zero_grad()
    loss.backward()
    optimizer.step()

with torch.no_grad():
    logits = model(X)
    probabilities = torch.sigmoid(logits)
    predictions = (probabilities >= 0.5).float()

print("Initial loss:", initial_loss)
print("Final loss:", loss.item())
print("Final probabilities:", probabilities)
print("Predicted labels:", predictions)
print("Gradient of first-layer weights:", model[0].weight.grad)
```

Task 4: Experimental Results

4A — Basic learning

Initial loss = 0.7482295; final loss = 0.00262987. Final probabilities were approximately 0.0033, 0.9976, 0.9976, and 0.0024 for (0,0), (0,1), (1,0), and (1,1), respectively. Thresholding at 0.5 produced [0, 1, 1, 0], giving 4/4 correct.

4B — Backpropagation

At the start of training, the first-layer gradient was $\begin{bmatrix} -0.0040, -0.0044, 0.0064, 0.0081 \end{bmatrix}$, with gradient norm 0.0118937. This nonzero gradient confirms that backpropagation produced a learning signal for the first-layer weights. The gradient represents $\partial L / \partial W^{(1)}$.

4C — Zero-initialization symmetry

All weights and biases were initialized to zero. The hidden-layer weights remained $[[0,0],[0,0]]$ through five training steps. Both hidden units therefore remained identical. In this particular setup, the output layer was also initialized to zero, causing zero gradient to reach the hidden layer initially.

4D — Activation comparison

Hidden activation	Final loss	Correct	Early gradient norm
Sigmoid	0.002630	4/4	0.011894
Tanh	0.000637	4/4	0.028684
ReLU	0.346600	3/4	0.082002

In this run, sigmoid and tanh learned XOR completely, while ReLU achieved 3/4 correct. ReLU had the largest early gradient norm, showing that a larger initial gradient does not by itself guarantee better final learning.

Task 5: Three-Class Extension

Classes: 0 = (0,0), 1 = disagreement [(0,1),(1,0)], and 2 = (1,1). The architecture was changed to $2 \rightarrow 2 \rightarrow 3$ logits and CrossEntropyLoss was used.

Input	Logit 0	Logit 1	Logit 2
(0,0)	9.4181	1.2406	-10.5668
(0,1)	-2.8013	6.7260	-2.2137
(1,0)	-2.8013	6.7260	-2.2137
(1,1)	-7.2236	0.4837	8.6185

Final loss = 0.00024554. Predicted classes = [0,1,1,2], matching the true classes exactly. Softmax probabilities summed to 1.0000 for all four examples.

For multiclass cross-entropy, the logit gradient is $\partial L / \partial z_i = p_i - y_i$. The correct class has $y_i = 1$, producing a negative gradient when its probability is below 1 and therefore encouraging its logit to increase. Incorrect classes have $y_i = 0$, so their positive gradients encourage their logits to decrease.

Final Three-Class Code

```
torch.manual_seed(42)

X_3class = torch.tensor([
    [0.0, 0.0], [0.0, 1.0],
    [1.0, 0.0], [1.0, 1.0]
])
y_3class = torch.tensor([0, 1, 1, 2])

model_3class = nn.Sequential(
    nn.Linear(2, 2),
    nn.Tanh(),
    nn.Linear(2, 3)
)

criterion_3class = nn.CrossEntropyLoss()
optimizer_3class = torch.optim.SGD(
    model_3class.parameters(), lr=1.0
)

for step in range(5000):
    logits = model_3class(X_3class)
    loss = criterion_3class(logits, y_3class)
    optimizer_3class.zero_grad()
    loss.backward()
    optimizer_3class.step()

with torch.no_grad():
    logits = model_3class(X_3class)
    probabilities = torch.softmax(logits, dim=1)
```

```
predictions = torch.argmax(probabilities, dim=1)
```

Reflection

1. Depth vs. nonlinearity

Depth means stacking multiple layers. If all layers are affine, they can be combined into one affine map. Therefore, depth alone cannot represent XOR; a nonlinear hidden activation is required.

2. Evidence of learning

The loss decreased from 0.7482 to 0.00263, all four binary XOR examples were classified correctly, and the early first-layer gradient norm was nonzero (0.01189).

3. Initialization and symmetry

Identical hidden parameters produce identical outputs and gradients, so identical units remain identical. In the zero-initialization experiment, the hidden weights stayed exactly zero because the output layer was also initialized to zero.

4. Activation effects

The activation affected both gradient magnitude and final learning. In this run, sigmoid and tanh achieved 4/4, while ReLU achieved 3/4. A larger early gradient did not guarantee better final performance.

5. Output/loss pairing

Binary classification used logits with BCEWithLogitsLoss and sigmoid only for reporting probabilities. Multiclass classification used multiple logits with CrossEntropyLoss and softmax for displayed probabilities.

6. Responsible LLM use

The LLM helped produce concise PyTorch boilerplate and implementation. The architecture, data, loss, outputs, gradients, and numerical results were checked against the laboratory requirements; responsibility for verification remained with the student.

7. Tests for scaling

Retain checks for input/target shapes, forward-pass dimensions, loss behavior, gradients, appropriate prediction metrics, probability constraints, reproducibility, and edge cases.

Submission Checklist

- Notebook/Python implementation.
- Task 1 problem specification.
- Task 2 model design.
- Exact LLM prompt and documented corrections/changes.
- Final binary XOR code.
- Final three-class extension code.
- Loss, prediction, gradient, symmetry, and activation results.
- Reflection answers.